An experiment on the Learning of Deterministic Finite Automata (DFA)

Chenyi Zhang and Jason Wenxuan Miao

October 2024

Abstract

This draft reports the current progress of an empirical study for learning deterministic finite automata (DFA) from a finite sample set. Each element in a training sample consists of a string and a single bit indicating whether the string is accepted or rejected by the DFA to be learned. Given this problem is NP-hard in general, in this preliminary study, we only applied DFA learning on small sized DFA of less than 10 states. We leave the development of more sophisticated DFA learning algorithms as a future work.

1 Introduction

Deterministic finite automaton (DFA) learning, a special case of grammatical inference problem, is the problem of constructing a finite automaton from a training sample. Such a sample often consists of a set of strings, each paired with a single bit indicating whether that string is in the language or not. DFA learning is first introduced by Gold [4], who proposed a framework that points out a path for several ways of learning a formal language, either from a sample of strings in or not in the language, or from a combination of samples together with a complete representation of the hypothesis language (e.g., the L^* algorithm of Angluin [1]). This paper follows the former setting. That is, we aim to learn a DFA from a sample of string in or not in the language. We further call the subset of a given sample in the target language as *positive strings*, and those strings in the sample that are not in the target language as *negative strings*.

Existing results for the heuristics approach. Gold has shown that in general the problem of identifying a minimum state DFA consistent with a presentation is NP-hard [5]. However, in the same paper the author has also claimed that it is possible to reconstruct a DFA if we carefully choose a set of training samples that are characteristic of the source language [5]. A few follow-up papers [2, 7] have defined polynomial time and data identifiability as extensions of Gold's characteristic sets of polynomial size. The Regular Positive

and Negative Inference (RPNI) algorithm [10, 3] is a polynomial time algorithm that is guaranteed to return the minimal DFA (called the canonical representation) if a characteristic set for the target DFA is given as samples. Trakhtenbrot and Barzdin's algorithm [13] (TB algorithm) is another early work that outputs a minimum consistent DFA if the sample set is *uniformly-complete*. (i.e., the sample strings cover all possible words up to a given length) The TB algorithm serves as a paradigm that starting from the prefix tree acceptor consisting of positively and negatively labelled training samples, it is possible to merge states in that prefix tree and derive a DFA that has fewer states than the prefix tree. Based on this methodology, two state-merge heuristic algorithms (EDSM [9] and parallel beam [8]) have been used to solve the Abbadingo challenge problems of DFA with up to 500 states with relatively sparse branching conditions.

Other works related to DFA learning. We are aware of those works that solve this type of learning problem by making a reduction to the propositional formula satisfiability problem [6, 15] (SAT). The DFA learning problem is also discussed in the PAC setting [14, 12, 11]. We plan to carry out a detailed comparison to these works regarding the efficient approaches for DFA learning that are different from the state-merge approach in the future.

Preliminaries. A deterministic finite automaton (DFA) can be defined as a quintuple of the form $\langle Q, q_0, \Sigma, \delta, \mathcal{F}\rangle$, where Q is a finite set of states, $q_0 \in Q$ is the start state, Σ is a finite set of alphabet, $\delta : Q \times \Sigma \to Q$ is a transition function, and $\mathcal{F} \subseteq Q$ a finite set of accept states. To ease the presentation, given $q_1, q_2 \in Q$ and $a \in \Sigma$, we sometimes write $q_1 \xrightarrow{a} q_2$ if $q_2 = \delta(q_1, a)$. Given $q, q' \in Q$ and $w = a_1 a_2 \dots a_n \in \Sigma^*$, we also write $q \xrightarrow{w} q'$ if there exists $q_1, q_2, \dots q_n \in Q$ with $q_i \xrightarrow{a_{i+1}} q_{i+1}$ for all $i = 1 \dots n-1$, $q \xrightarrow{a_1} q_1$ and $q_n = q'$. Clearly, $L(\mathcal{A})$, the language accepted by a DFA $\mathcal{A}$, is given by $\{w \in \Sigma^* \mid \exists s \in \mathcal{F} : s_0 \xrightarrow{w} s\}$, i.e., the set of *words* (or strings) that possibly lead to a state in $\mathcal{F}$. The empty word is denoted by λ.

A training set $S \subseteq \Sigma^*$ is always equipped with a mapping $\ell : S \to \{1, 0\}$, such that given $w \in S$, $\ell(w)$ indicates whether or not w belongs to the language to be learned. Formally, define $D_+(S) = \{s \in S \mid \ell(s) = 1\}$ and $D_-(S) = \{s \in S \mid \ell(s) = 0\}$, and we write D_+ for $D_+(S)$ and D_- for $D_-(S)$ if it is clear from the context. We write $D_?$ for $\Sigma^* \setminus S$, or $\ell(w) =?$ if $w \notin S$. Usually, it is only interesting to talk about a word w not in S if w is a prefix of a word in S.

Let S be a training set equipped with ℓ, an automaton $\mathcal{A}$ is *consistent* with S if it accepts all words in D_+ and rejects all words in D_-. A minimal consistent DFA with respect to S is a DFA of n states which is consistent with S, and moreover, all other DFA consistent with S may have at least n states. Given a training set, there may be more than one minimal consistent DFA as shown in the following example.

Example 1 *Let we have the training set $D_+ = \{aa\}$ and $D_- = \{\lambda, a, ab\}$. We may find two DFA, as shown in Figure 1, both of which accept the word aa and*

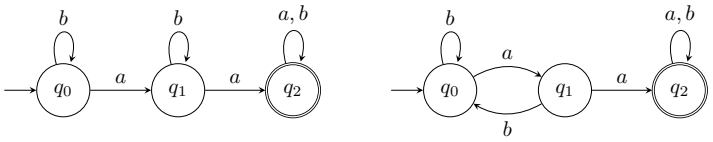

Figure 1: An example of two minimal DFA both consistent with a training set.

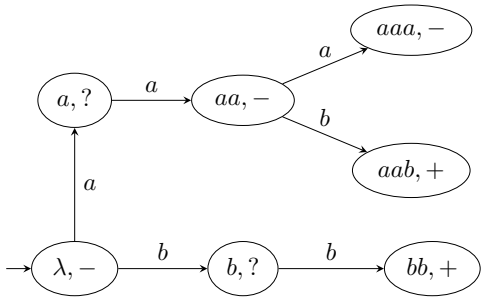

Figure 2: A Moore-machine from a training set.

reject λ, a, and ab. Both DFA are of least possible number of states. However, they disagree on the acceptance of the word aba.

2 Our approach

Often the first step to construct a DFA from a training sample is by treating the sample S as a *tree acceptor*, i.e., ad eterministic tree-shaped Moore-machine $T_S = \langle S^+, \lambda, \Sigma, \delta, \ell, \{0, 1, ?\}\rangle$, where S^+ is the prefix closure of S, λ is the initial state, $\delta(w, a) = wa$ if $w, wa \in S^+$. The observation ℓ maps w to '+' if $w \in D_+$, or to '−' if $w \in D_-$, or '?' otherwise. Figure 2 shows a tree acceptor that represents the training set S with $D_+ = \{aab, bb\}$ and $D_- = \{\lambda, aa, aaa\}$. Since the words a and b are not in S, we have $\ell(a) = \ell(b) =$'?' in the figure.

The existence of "?" poses a challenge in generating a DFA by folding the tree acceptor. A tree-folding procedure usually starts by trying to find different nodes in the tree that can be safely merged without causing inconsistency to the training set S. However, if S is not *prefix closed*, then there exists some $w_1w_2 \in S$ with $w_2 \neq \lambda$ such that $w_1 \notin S$. This w_1 produces a hole (marked by "?", shown in Figure 2) in the tree acceptor, introducing uncertainties when the nodes in the tree are evaluated before the merge, just as the first proof showing that DFA-learning with at most n states is NP-hard [5] takes advantage of those "?", or holes, in the tree acceptor. A later work has shown that finding minimal state DFA is still NP-hard even if a tree acceptor has no hole [16]. Equivalently, this is when the training set S is prefix-closed.

Algorithm 1 Deterministic greedy merge

Input: S^+ is the prefix closure of $D_+ \cup D_-$, $weight : S^+ \to \mathbb{N}$ the weight function, and $comp \subseteq S^+ \times S^+$ the compatibility function
Output: A set of equivalence classes
Initialize: $eq := \{w \mapsto \{w\} \mid w \in S^+\}$, $node\_list := list(S^+)$

```
sort node_list by reversed weight
while node_list ≠ [] do
    if len(node_list = 1) then break
    end if
    state_one = node_list[0]; idx := 1
    while not comp(state_one, node_list[idx] and idx < len(node_list) do
        idx := idx + 1
    end while
    if idx = len(node_list) then remove
        state_one from node_list
        continue
    end if
    merge(eq(state_one), eq(node_list[idx]))
    removed node_list recursively
    update merge
end while
return eq (* the new collection of equivalence classes *)
```

In this preliminary study, our approach is a deterministic state-merge algorithm that learns a DFA from a training set S in the following steps. In this algorithm, we assume that the alphabet is $\Sigma = \{0, 1\}$.

1. **Construct a tree acceptor.** We read a set of strings from a training sample S and create a tree acceptor that is similar to what is shown in Figure 2. For each node $w \in S^+$ in the tree acceptor, there exists $w' \in S$ such that w is a prefix of w'. A node is labeled $(w, +)$ if $w \in D_+$, it is labeled $(w, -)$ if $w \in D_-$, and $(w, ?)$ if $w \notin S$.

2. **Compute a compatibility relation on the tree node.** Given $w_1, w_2 \in S^+$, w_1 and w_2 are compatible if (1) whenever $w_1 \in S$ and $w_2 \in S$, they must be both labeled + or both labelled −, and (2) each subtree of w_1 is compatible with the correponding subtree of w_2 if both subtrees exist for that extension. For example, given the alphabet $\{0, 1\}$, we require $w_1 0$ compatible with $w_2 0$ if we have $w_1 0 \in S^+$ and $w_2 0 \in S^+$, and $w_1 1$ compatible with $w_2 1$ if we have both $w_1 1 \in S^+$ and $w_2 1 \in S^+$. If $w_i 0$ is missing for w_i then the 0-th extension is satisfied for checking compatibility of w_1 and w_2. In the implementation, we used dynamic programming to compute the relation $comp \subseteq S^+ \times S^+$.

3. **Merge nodes in the tree.** The algorithm is presented in Algorithm 1. When merging w_1 with w_2, we also merge $w_1 0$ and $w_2 0$ if both nodes have

a 0 successor, and similarly for 1 successors. After merging w_2 with w_1, we remove w_2 from the list. Since it is possible that there may exist some w' that is compatible with w_1 but incompatible with w_2, we need to update the relation *comp* as w' should be treated as incompatible with w_1. This is because after the merge of w_1 and w_2, node w' will become incompatible with $eq(w_1)$ given $w_2 \in eq(w_1)$. If after a merge, the equivalence class represented by $eq(state\_one)$ becomes incompatible with all remaining states in *node_list*, we will remove *state_one* from the list, and resume the merge process starting from the first node in the new *node_list*.

4. **Output a DFA.** After merging the tree nodes, we have generated a collection of equivalence relations on S^+. Given an insufficient sample, we may produce an incomplete DFA in the following ways. (1) Some states $eq(w)$ (equivalence classes) are not labeled, and (2) a transition for some state $eq(w)$ and action a (from the alphabet Σ) is undefined. We analyze the possible scenarios in the following paragraph.

An output DFA. Let Q be the set of equivalence classes which is an output of Algorithm 1. We should have $\bigcup Q = S^+$ and $eq(w) \in Q$ for all $w \in S^+$. For the complete DFA of the form $\langle Q, q_0, \Sigma, \delta, \mathcal{F} \rangle$, we have

- $q_0 = eq(\lambda)$,
- $\mathcal{F} = \{eq(w) \mid w \in D_+\}$,
- $\delta(eq(w), a) = eq(w')$ if there exists $w'' \in eq(w)$ such that $w''a = w'$.

Now consider the following scenario.

- There exists $w \in S^+$, for all $w' \in eq(w)$, $w' \in S^+ \backslash (D_+ \cup D_-)$. That means, all states merged into $eq(w)$ is not defined with accept/non-accept, i.e., none of these states is in S.
- There exists $w \in S^+$ and $a \in \Sigma$, such that $\delta(eq(w), a)$ is undefined. That means, for all states w' merged into $eq(w)$, $w'a$ is not in S^+.

3 Experiment

We have written a simple DFA generation algorithm in python, which generates a random DFA of a given size. The algorithm then simplify the DFA to minium size, and compute the depth of that minimum DFA. The depth of state q in a DFA is the minimum number of transitions from q_0 to reach q, and the depth of a DFA is the depth of a state that has the largest depth.

Environment. The experiment is carried out in a Linux Mint 21.1 Cinnamon desktop (version 5.15.0-91-generic) on the hardware of Intel Core i7-9700 CPU @ 3.00GHz (8 Core) 32GB memory with light load. The algorithm has taken

case	#states	depth	t-size	tree-size	o-DFA	m-label	m-trans
0	7	4	97	118	20	0	1
1	9	5	194	237	12	0	0
2	8	4	101	119	16	0	0
3	9	4	95	119	20	0	0
4	9	7	984	1600	137	0	0

Table 1: The result of our preliminary experiment

roughly two weeks' working time to implement, with the python source code uploaded to github[1].

We have generated five minimum-state DFA models of 7-9 states of different levels of edge complexities to test the deterministic greedy algorithm. The runtime of algorithm on each model takes in general less than 1 second. However, due to the NP-hardness nature of the problem, the algorithm does not find the minimal DFA in all 5 cases. The result of the experiment is presented in Table 1. For example, the randomly generated DFA case 0 is the easiest case, which has 7 states with depth 4. The provided sample size is 97 (strings each with a boolean indicating accept or not). The deterministic algorithm first generates an acceptor tree of size 118 (which is S^+ in the definition of Section 2), and then condense the tree into a DFA of 20 states (instead of 7 states as the original DFA). All states of the generated DFA are labeled by either accept or non-accept, though there is one missing transition. The cause of that missing transition may be due to the algorithm, or insufficient size of the training sample. The algorithm makes a better DFA from case 1, probably due to the relatively larger size of the training sample.

4 Conclusion

This paper only serves as a plan with some preliminary results. The initial result is not ideal since the generated DFA often has a much larger size than the original DFA that is used to generate the training sample. Our next steps include exploration on (1) the use of ranking function similar to those adopted in [9, 8] and (2) randomization in state-merging attempts.

References

[1] D. Angluin. Learning regular sets from queries and counterexamples. *Information and Computation*, 75:87–106, 1987.

[2] D. Angluin and C. H. Smith. Inductive inference: Theory and methods. *ACM Computing Surveys*, 15(3):237–269, 1983.

[1] Available at `https://github.com/czhang-fm/dfa-learning-experiment`

[3] P. García, D. López, and M. Vázquez de Parga. Polynomial characteristic sets for DFA identification. *Theoretical Computer Science*, 448:41–46, 2012.

[4] Mark E. Gold. Language identification in the limit. *Information and Control*, 10:447–474, 1967.

[5] Mark E. Gold. Complexity of automaton identificaton from given data. *Information and Control*, 37:302–320, 1978.

[6] Marijn Heule and Sicco Verwer. Exact DFA identification using SAT solvers. In *Proc. Grammatical Inference: Theoretical Results and Applications, ICGI 2010*, pages 66–79, 2010.

[7] C. De La Higuera. Characteristic sets for polynomial grammatical inference. *Machine Learning*, 27(2):125–138, 1997.

[8] H. Juillé and J. B. Pollack. A stochastic search approach to grammar induction. In *Proc. International Colloquium on Grammatical Inference (ICGI'98)*, pages 126–137, 1998.

[9] K. J. Lang, B. A. Pearlmutter, and R. A. Price. Results of the abbadingo one DFA learning competition and a new evidence-driven state merging algorithm. In *Proc. International Colloquium on Grammatical Inference (ICGI'98)*, pages 1–12, 1998.

[10] J. Oncina and P. García. Inferring regular languages in polynomial update time. *Pattern recognition and image analysis, World Scientific*, 1:49–61, 192.

[11] R. Parekh and V. Honavar. Learning dfa from simple examples. *Machine Learning*, 44(1-2):9–35, 2001.

[12] L. Pitt and M. K. Warmuth. The minimum consistent dfa problem cannot be approximated within any polynomial. *Journal of the ACM*, 40(1):95–142, 1993.

[13] B. A. Trakhtenbrot and Y. M. Barzdin. *Finite automata; behavior and synthesis (Fundamental studies in computer science)*. North-Holland Pub. Co., 1973.

[14] L. Valiant. A theory of the learnable. *Communications of the ACM*, 27:1134–1142, 1984.

[15] Ilya Zakirzyanov, António Morgado, Alexey Ignatiev, Vladimir Ulyantsev, and João Marques-Silva. Efficient symmetry breaking for sat-based minimum DFA inference. In *LATA*, volume 11417 of *Lecture Notes in Computer Science*, pages 159–173, 2019.

[16] Chenyi Zhang. Minimal consistent DFA from sample strings. *Acta Informatica*, 57(3-5):657–670, 2020.